

勤怠入力機能で学ぶ 用語まとめ(第 3 章)

～ 計画・実装・検証から、エラー処理・テスト・デプロイまで ～

この文書は、勤怠入力(TimeEntry)機能づくりでよく出てくる言葉を、身近な例えとセットで、知識ゼロから分かるようにまとめたものです。

作成日:2026 年 6 月 17 日

はじめに — この文書の使い方

この文書は、用語の意味を初学者向けにやさしくまとめたものです。むずかしい言葉は、レストランや倉庫などの身近なものにたとえて説明しています。

各用語は「ひとことで言うと → 身近な例え → 具体例 → つながり」の同じ型で並べています。まずは「ひとことで言うと」だけ拾い読みしても大丈夫です。

◆ この文書で扱う用語(テーマ別)

- **開発の進め方:** Plan mode / TDD / リファクタリング
- **コードの部品と役割分担:** メソッド / TimeEntryController / TimeEntryService / ビジネスロジック
- **API 入力の受け取りと検証:** リクエストパラメータ / API のバリデーション不足 / 型チェック
- **エラー処理:** エラーハンドリング / try-catch / カスタム例外クラス / IllegalArgumentException
- **TimeEntry の操作(CRUD):** createTimeEntry / updateTimeEntry / deleteTimeEntry / calculateMonthlyTotal / トランザクション
- **品質を守る:** エッジケース / テストカバレッジ / デプロイ

第 1 部 — 開発の進め方

1. Plan mode(プラン モード)

ひとことで言うと:いきなり作り始めず、まず「何をどう直すか」の計画だけを立てる Claude Code の作業モードのこと。

旅行でいきなり出発せず、先に行き先と回る順番を地図で決めてから動くようなものです。実際の変更は加えず、段取りを先に固めるので、後戻りが減ります。

◆ Plan mode のうれしさ

- 勝手にコードを書き換えないので安心して相談できる
- 計画に納得してから実装を始められる
- やることの全体像が先に見える

【つながり】 計画ができたなら実装に入りますが、その実装の質を上げる進め方が、後で出てくる「TDD」です。

2. TDD(ティーディーディー / Test-Driven Development)

ひとことで言うと:先にテスト(合格条件)を書いてから、それに通るコードを書いていく開発の進め方のこと。

料理の前に「味の合格ライン」を先に決めておき、その味見をクリアするように作っていくようなものです。ゴールを先に置くので、作りすぎや作り忘れが減ります。

◆ TDD の進め方(イメージ)

1. まず失敗するテストを書く(赤)
2. テストに通る最小のコードを書く(緑)
3. 動いたまま中身をきれいにする(リファクタリング)

【つながり】 3 番目の「動いたまま中身をきれいにする」が、次の「リファクタリング」です。

3. リファクタリング(Refactoring)

ひとことと言うと:外から見た動きは変えずに、コードの中身だけを整理して読みやすくすることのこと。

料理の味はそのままに、**散らかった厨房を片付けて道具を並べ直すようなもの**です。できあがり(動き)は同じでも、次に作る人がぐっと楽になります。

◆ よくやる整理

- 長すぎる処理を、意味のあるまとまりに分ける
- 分かりにくい名前を、分かる名前に変える
- 同じ処理の重複を 1 か所にまとめる

【つながり】 「意味のあるまとまりに分ける」ときの、その 1 つのまとまりが、次の「メソッド」です。

第2部 — コードの部品と役割分担

4. メソッド(Method)

ひとことと言うと:ひとままとりの処理に名前を付けて、何度でも呼び出せるようにした「ひとつの作業」のこと。

「コーヒーを淹れる」という一連の手順に名前を付けた、**ボタン一つで動く家電のようなもの**です。中の手順を毎回書かなくても、名前を呼ぶだけで同じ作業ができます。

◆ 書き方(イメージ)

```
// 「月の合計を出す」作業に名前を付ける
int calculateMonthlyTotal(...) {
    ...一連の計算...
}
```

【つながり】 このメソッドたちを役割ごとにまとめた部品が、次の「TimeEntryController」や「TimeEntryService」です。

5. TimeEntryController(タイムエントリー コントローラー)

ひとことと言うと:勤怠入力「窓口」として、外からのお願いを受け取って中へ取り次ぐ部品のこと。

レストランの注文を受けるウェ이터のようなものです。お客さん(画面)からの注文を受け取り、厨房(後ろの処理)へ渡し、できた料理を返します。自分では調理しません。

◆ Controller の役割

- 画面からのリクエストを受け取る
- 受け取った内容を中の処理担当へ渡す
- 結果を画面に返す(調理そのものはない)

【つながり】 ウェ이터が厨房へ渡す先、つまり実際に調理する処理担当が、次の「TimeEntryService」です。

6. TimeEntryService(タイムエントリー サービス)

ひとことと言うと:勤怠入力の中身の処理(計算や判断)を実際に行う「厨房」にあたる部品のこと。

注文を受け取って実際に料理を作る厨房のようなものです。窓口(Controller)から渡された依頼を受け、登録・計算・チェックなどの本当の仕事をします。

◆ Service が担当すること

- 勤怠データの登録・更新・削除
- 月の合計時間の計算
- 「業務上やってよいか」の判断

【つながり】 この厨房で行う「業務上やってよいかの判断」のルールそのものが、次の「ビジネスロジック」です。

7. ビジネスロジック(Business Logic)

ひとことと言うと:そのアプリの「業務上の決まりごと」を、処理として表したルールのこと。

レストランの「ラストオーダーは 21 時まで」「1 人 3 点まで」といった、**お店ならではの決まりのようなもの**です。仕組みの良し悪しではなく、業務として正しいかを決めます。

◆ 勤怠でのビジネスロジック例

- 退勤時刻は出勤時刻より後でなければならない
- 1 日の勤務は 24 時間を超えてはいけない
- 未来の日付では打刻できない

【つながり】 このルールに反する入力をはじく入口の守りが、次の「リクエストパラメータ」の受け取りと検証です。

第3部 — API 入力の受け取りと検証

8. リクエストパラメータ(Request Parameter)

ひとことで言うと:画面やアプリから送られてくる「お願いの中身」(日付や時刻などの値)のこと。

レストランの注文票に書かれた「料理名」「数量」のようなものです。勤怠入力なら「誰が・いつ・何時から何時まで」といった値が、これにあたります。

◆ 勤怠入力のパラメータ例

- 従業員 ID(誰の勤怠か)
- 勤務日(いつの勤怠か)
- 出勤時刻・退勤時刻

【つながり】 この注文票の中身を確認めずに通してしまう手抜きが、次の「API のバリデーション不足」です。

9. API のバリデーション不足(Validation 不足)

ひとことで言うと:送られてきた値を十分に確認しないまま受け入れてしまい、おかしいデータが通る状態のこと。

注文票の中身を確認めず、空欄や落書きのままでも厨房へ流してしまう受付のようなものです。後の工程で必ず事故になり、原因も追いにくなります。

◆ チェック漏れで起きること

- 必須項目が空のまま登録されてしまう
- 退勤が出勤より前など、ありえない値が通る
- おかしなデータが後の計算を壊す

【つながり】 この不足を防ぐ第一歩、「値の形が正しいか」を確認めるのが次の「型チェック」です。

10. 型チェック(Type Check)

ひとことで言うと:受け取った値が「期待した種類(数値・日付など)」になっているかを確認することのこと。

郵便局で「これは封筒」「これは小包」と種類ごとに分ける仕分け作業のようなものです。日付の欄に文字が入っていないか、数値の欄に数字が入っているかを先に確かめます。

◆ 型チェックの例

勤務日 → 日付の形か?(2026-06-17 など)
出勤時刻 → 時刻の形か?
従業員 ID → 数値か?(文字が混ざっていないか)

【つながり】 形が違う・値がおかしいと分かったとき、それを知らせる代表的な例外が次の「IllegalArgumentException」…の前に、まず例外の扱い方「エラーハンドリング」を見ます。

第 4 部 — エラー処理

11. エラーハンドリング(Error Handling)

ひとことで言うと:エラーが起きたときに、慌てず受け止めて適切に対応する処理のこと。

ビルの防災センターのようなものです。火災報知器(エラー)が鳴ったら、放置せず、状況を確認かめて避難誘導や消火など決められた対応をします。

◆ エラーハンドリングですること

- エラーを受け止めて、アプリが落ちないようにする
- 利用者に分かるメッセージを返す
- 原因を追えるよう記録を残す

【つながり】 このエラーを実際に受け止める基本の書き方が、次の「try-catch」です。

12. try-catch(トライ キャッチ)

ひとことで言うと:「やってみる(try)」処理と、「失敗したら受け止める(catch)」処理をセットで書く仕組みのこと。

綱渡りの下に張っておく安全ネットのようなものです。try で挑戦し、もし落ちても catch のネットが受け止めるので、全体が重大事故になりません。

◆ 書き方(イメージ)

```
try {
    勤怠を登録する           // やってみる
} catch (例外 e) {
    エラーとして処理する    // 失敗したら受け止める
}
```

【つながり】 この catch で受け止める「失敗のしるし」を、自分たちの業務に合わせて作るのが次の「カスタム例外クラス」です。

13. カスタム例外クラス(Custom Exception)

ひとことと言うと:標準の例外では足りないとき、自分たちのアプリ専用にするオリジナルのエラーの型のこと。

既製の札では足りず、「退勤が出勤より前です」と手書きの専用名札を用意するようなものです。何が問題かを業務の言葉で正確に伝えられます。

◆ 作り方(イメージ)

```
// 勤怠専用のエラーを定義する
class InvalidTimeEntryException extends RuntimeException {
    ...
}
```

【つながり】 一方、Java が最初から用意している代表的な標準例外が、次の「IllegalArgumentException」です。

14. IllegalArgumentException(イリーガル アーギュメント エクセプション)

ひとことと言うと:メソッドに渡された値が不正なとき、Java が標準で投げしてくれる定番のエラーのこと。

受付で「その申込書の書き方はおかしいので受け取れません」と突き返されるようなものです。引数(渡された値)が約束を守っていないと、はっきり知らせます。

◆ 投げる場面の例

```
if (退勤 < 出勤) {
    throw new IllegalArgumentException("退勤は出勤より後にしてください");
}
```

【つながり】 こうした検証や例外を実際に行いながらデータを登録するのが、次の「createTimeEntry」です。

第 5 部 — TimeEntry の操作(CRUD)

15. createTimeEntry(クリエイト タイムエントリー)

ひとことと言うと:新しい勤怠データを 1 件「作って登録する」メソッドのこと。

ノートに新しい 1 行を書き足すようなものです。出勤・退勤などの値を受け取り、検証してから新しい勤怠記録として保存します。

◆ create の流れ

- リクエストパラメータを受け取る
- 値を検証する(おかしければ例外)
- 新しい勤怠データとして保存する

【つながり】 一度作った記録をあとから直すのが、次の「updateTimeEntry」です。

16. updateTimeEntry(アップデート タイムエントリー)

ひとこと言うとは:すでにある勤怠データの中身を「書き換えて更新する」メソッドのこと。

ノートにすで書いた行を、消しゴムで消して書き直すようなものです。打刻ミスの修正など、既存の 1 件を新しい値に上書きします。

◆ update の流れ

- どの記録を直すかを指定する(ID など)
- 新しい値を検証する
- 見つかった記録を上書き保存する

【つながり】 書き換えではなく、記録そのものを消すのが、次の「deleteTimeEntry」です。

17. deleteTimeEntry(デリート タイムエントリー)

ひとこと言うとは:いらなくなった勤怠データを 1 件「消す」メソッドのこと。

ノートの1行を、まるごと消しゴムで消すようなものです。誤って登録した記録などを、指定して削除します。元に戻せないことが多いので慎重に行います。

◆ delete で気をつけること

- 消してよい記録かを先に確かめる
- 存在しない記録の削除は例外で知らせる
- 消した後は元に戻せないことが多い

【つながり】 これら作る・直す・消すの記録をもとに、月単位で集計するのが、次の「calculateMonthlyTotal」です。

18. calculateMonthlyTotal(カリキュレート マンスリー トータル)

ひとことで言うと:1 か月分の勤怠データを合計して、月の勤務時間を計算するメソッドのこと。

家計簿で1 か月分を月末にまとめる集計作業のようなものです。日々の勤怠を1 件ずつ足し合わせ、その月の合計勤務時間を出します。

◆ 計算の流れ

1. その月の勤怠データを全部集める
2. 1 件ずつ勤務時間を足していく
3. 合計を返す

【つながり】 作る・直す・消す・集計する一連の処理を「途中で失敗したら全部なかったことにする」安全装置が、次の「トランザクション」です。

19. トランザクション(Transaction)

ひとことで言うと:複数の処理を「全部成功か、全部やり直し」のひとまとまりとして扱う仕組みのこと。

銀行の振込で「相手に入金」と「自分から引き落とし」を、どちらか片方だけ成功させないための、まとめて1 回の手続きのようなものです。途中で失敗したら、全部なかったことに戻します。

◆ トランザクションの考え方

- 途中まで成功・途中から失敗、という中途半端を防ぐ
- 失敗したら、最初の状態に巻き戻す(ロールバック)
- 全部そろって成功したときだけ確定する

【つながり】 こうした処理が「あらゆる入力で正しく動くか」を試すとき、特に注意するのが次の「エッジケース」です。

第 6 部 — 品質を守る

20. エッジケース(Edge Case)

ひとことで言うと:めったに起きないが、起きると失敗しやすい「端っこの特殊な条件」のこと。

遊園地の身長制限の看板で、ちょうど境目の人を見るように、**ふだん来ない「ぎりぎりの状況」を想像してみるようなもの**です。普通の入力では問題なくとも、端っこの値でバグが出やすいのです。

◆ 勤怠でのエッジケース例

- 出勤と退勤が同じ時刻(勤務 0 分)
- 日付をまたぐ深夜勤務(0 時をまたぐ)
- うるう年の 2 月 29 日

【**つながり**】 こうした特殊条件まで含めて「どれだけ試せているか」を測る物差しが、次の「テストカバレッジ」です。

21. テストカバレッジ(Test Coverage)

ひとことで言うと:コードのうち、どれだけの範囲がテストで試されているかを表す割合のこと。

試験勉強で、教科書の何割を解き終えたかを示す進捗バーのようなものです。割合が高いほど、試していない「穴」が少ない目安になります(高ければ必ず安全、ではありません)。

◆ カバレッジの見かた

割合の状態	意味	目安
低い	試せていない部分が多い	穴が多く不安
高い	多くの部分を試している	ただし中身も大事
100%でも	全部通ったとは限らない	条件の組合せに注意

【つながり】 テストで品質を確かめたら、いよいよ本番の環境へ届ける、次の「デプロイ」へ進みます。

22. デプロイ (Deploy)

ひとことで言うと:作って試したアプリを、実際に使われる本番の環境へ配置して動かすことのこと。

完成した料理を、厨房から客席へ運んで提供するようなものです。手元で動くだけでなく、本番の場所に置いて初めて、利用者が使えるようになります。

◆ デプロイで気をつけること

- テストに通ってからデプロイする
- 問題があればすぐ前の状態へ戻せるようにする
- 利用者への影響が少ない時間帯を選ぶ

【つながり】 デプロイ後に問題が見つかれば、また「Plan mode」で直し方を計画する、最初の一步へ戻ります。

付録 A — 全体まとめ表

用語	ひとことと言うと	身近な例え
Plan mode	先に計画だけ立てる作業モード	地図で順番を決めてから動く
TDD	テストを先に書く開発の進め方	先に味の合格ラインを決める
リファクタリング	動きを変えず中身を整理	味はそのままに厨房を片付ける
メソッド	名前を付けた一つの作業	ボタン一つで動く家電
TimeEntryController	お願いを受け取る窓口	注文を受けるウェイター
TimeEntryService	中身の処理を行う担当	実際に料理する厨房
ビジネスロジック	業務上の決まりごと	店の「ラストオーダー」ルール
リクエストパラメータ	送られてくるお願いの中身	注文票の料理名・数量
API のバリデーション不足	値を確かめず通す手抜き	注文票を確かめず厨房へ
型チェック	値の種類が正しいか確認	封筒か小包かの仕分け
エラーハンドリング	エラーを受け止めて対応	ビルの防災センター
try-catch	挑戦と失敗受け止めのセット	綱渡りの安全ネット
カスタム例外クラス	業務専用の自作エラー型	手書きの専用名札
IllegalArgumentException	不正な値の標準エラー	書き方が変で突き返す受付
createTimeEntry	勤怠を作って登録	ノートに新しい行を書く
updateTimeEntry	勤怠の中身を更新	書いた行を消して書き直す
deleteTimeEntry	勤怠を 1 件消す	ノートの行をまるごと消す
calculateMonthlyTotal	月の勤務時間を合計	家計簿の 1 か月分を合計

トランザクション	全部成功か全部やり直し	振込のまとめて1回の手続き
エッジケース	起きると失敗しやすい端の条件	身長制限ちょうど境目の人
テストカバレッジ	テストで試した範囲の割合	教科書の進捗バー
デプロイ	本番環境へ配置して動かす	料理を客席へ運んで提供

おわりに — 次の一歩

知らない言葉も、身近な例えとセットで覚えると、ぐっと頭に入ります。分からなくなったら、何度でも読み返してください。

◆ こんな進め方がおすすめ

- 「ひとことで言うと」だけ先に通して読む
- 身近な例えとセットで思い出す
- つながりをつたって、関係する用語をまとめて覚える

—— あなたのペースで大丈夫。応援しています! ——